

Microcontroladores

Roteiro 3 — Display LCD alfanumérico

Raoni F. S. Teixeira · Rodolfo Quadros · DENE/UFMT · 1 sessão · 1,0 ponto · grupos de 3

Objetivos desta sessão

Ao final desta sessão o grupo deve ser capaz de:

1. Inicializar um display HD44780 16x2 em barramento de 8 bits;
2. Distinguir **comando** de **dado** e explicar o papel do pino RS;
3. Respeitar os tempos de resposta de um periférico externo;
4. Converter valores inteiros em caracteres para exibição decimal.

1 Configuração de bancada

Configuração de bancada

CH2-1 (LCD)	ON
Chaves SWITCHS	Todas em OFF
CH3-3 e CH3-5	OFF

O contraste é ajustado pelo **trimpot R37**. Se o display acender sem mostrar nada, ou mostrar apenas quadrados escuros, ajuste R37 **antes** de suspeitar do código.

2 Fundamento

2.1 O barramento é compartilhado

No XM118, os dados do LCD usam o **mesmo PORTD dos 8 LEDs**. Escrever no display faz os LEDs piscarem.

Observação

Isso é comportamento esperado, não defeito. E ilustra um problema real de projeto: pinos são recurso escasso, e o compartilhamento tem consequências visíveis.

2.2 Ligação

Sinal	Pino do PIC
Dados D0 a D7	PORTD
RS (<i>Register Select</i>)	RE0
E (<i>Enable</i>)	RE1

Tabela 1: Ligação do display alfanumérico no XM118.

2.3 Comando ou dado

O pino **RS** decide como o display interpreta o byte presente no barramento:

- RS = 0 → **comando** (limpar, posicionar cursor, configurar);
- RS = 1 → **dado** (caractere a exibir).

O pino **E** é o sinal de validação: o display só lê o barramento na transição de E. Sem o pulso, nada acontece — o dado fica nos fios e é ignorado.

2.4 O display é lento

Este é o primeiro periférico da disciplina que **não obedece instantaneamente**. Cada operação exige tempo de processamento interno, e alguns comandos são bem mais lentos que outros.

Operação	Tempo mínimo
Comando comum	≈ 40 μs
Limpar display (0x01)	≈ 1,6 ms
Retornar cursor (0x02)	≈ 1,6 ms
Energização inicial	≈ 40 ms

Tabela 2: Tempos de resposta do controlador HD44780.

Atenção

Ignorar esses tempos produz o sintoma mais traiçoeiro do roteiro: o display funciona **às vezes**.

3 Parte 1 — Estudo do driver

O professor fornece `lcd.c` e `lcd.h`. **Leia antes de usar.**

1.1 Localize a função `lcd_pulso_enable()`. Por que existem dois atrasos, um antes e outro depois de baixar o pino E?

1.2 Em `lcd_comando()` há um teste condicional:

```
if (c == 0x01u || c == 0x02u) {
    __delay_ms(2);
}
```

Por que apenas esses dois comandos recebem espera adicional?

1.3 Compare `lcd_comando()` e `lcd_caractere()`. Qual é a **única** diferença funcional entre as duas?

Previsão — registre ANTES de gravar

1.4 — Em `lcd_iniciar()`, o que aconteceria se o `__delay_ms(50)` inicial fosse removido?

Formule a hipótese **antes** de testar. O teste ocorre no item 2.4.

4 Parte 2 — Primeiros textos

Tarefa

2.1 — Escreva um programa que mostre, ao ligar:

Grupo NN
Roteiro 3

com o número do seu grupo.

Tarefa

2.2 — Localize a sequência de inicialização e identifique o que cada comando configura. Consulte a tabela de comandos do HD44780 em anexo.

Comando	Função
0x38	
0x0C	
0x06	
0x01	

2.3 Troque 0x0C por 0x0F e observe. O que mudou?

Tarefa

2.4 — Remova o `__delay_ms(50)` inicial e grave.

O display funciona? Funciona **sempre**, ou apenas às vezes? Ligue e desligue o kit três vezes e anote cada resultado.

Tentativa 1	
Tentativa 2	
Tentativa 3	

Conceito central

Este teste é o ponto central do roteiro.

Um sistema que funciona “quase sempre” é mais perigoso que um que nunca funciona: o defeito não aparece na bancada, aparece em campo — e sob condições que ninguém consegue reproduzir.

Confronte agora o resultado com a sua previsão do item 1.4.

5 Parte 3 — Números no display

O display recebe **caracteres**, não números. Para mostrar o valor 37 é preciso enviar o caractere '3' e depois o '7'.

A conversão de dígito para caractere usa a tabela ASCII:

```
caractere = '0' + digito;
```

3.1 Por que essa soma funciona? O que garante que os dígitos de '0' a '9' sejam consecutivos na tabela?

Tarefa

3.2 — Escreva uma função que mostre um valor de 0 a 255 em decimal.

```
void lcd_numero(uint8_t v)
{
    /* completar */
}
```

Dica: os operadores / e % isolam cada dígito.

Tarefa

3.3 — Use a função para exibir um contador de 0 a 255, incrementando a cada 500 ms.

3.4 Ao passar de 100 para 99, o que aparece no display? Explique o que acontece e proponha uma correção.

6 Parte 4 – Integração com o roteiro anterior

Tarefa

4.1 – Combine com o Roteiro 2: mostre no LCD o **valor decimal** das chaves DIP, mantendo os LEDs espelhando o estado em binário.

Resultado esperado: chaves em 00000101 produzem 5 no display e os LEDs correspondentes acesos.

4.2 Você notou os LEDs piscando de forma estranha ao atualizar o display? Explique por quê, com base no fundamento desta sessão.

7 Teste de aceitação

- ☐ Display mostra as duas linhas de identificação do grupo.
- ☐ Contador decimal de 0 a 255 funcionando, sem resíduo de dígito.
- ☐ Valor decimal das chaves no LCD e binário nos LEDs (item 4.1).

8 Entrega e critério

Peso	Item
0,2	Teste de aceitação aprovado
0,2	Respostas 1.1 a 1.4
0,2	Tabela 2.2 completa
0,2	Observação 2.4 com as três tentativas registradas
0,2	Respostas 3.4 e 4.2

Tabela 3: Distribuição do ponto do Roteiro 3.

9 Armadilhas frequentes

Sintoma	Causa provável
Display aceso, sem texto	Contraste (R37) ou CH2-1 em OFF
Linha superior com quadrados escuros	Inicialização incompleta
Funciona às vezes	Falta de atraso após a energização
Caracteres embaralhados	Falta de espera após o comando 0x01
Texto na linha errada	Endereço de cursor: linha 0 = 0x80, linha 1 = 0xC0
Resíduo ao diminuir dígitos	Não limpou a posição anterior

Tabela 4: Diagnóstico rápido do Roteiro 3.

10 Para a próxima sessão

Tarefa

O kit tem um sensor de temperatura LM35, que entrega **10 mV por grau Celsius**. O microcontrolador lê tensão como um número inteiro de 0 a 1023, onde 1023 corresponde a 5 V.

Traga respondido, com a conta desenvolvida: se a leitura for **512**, qual a temperatura?